

Javascript_Day5_Handson_Gurunath_Rageni

Problem 1: Greeting Generator (Level-1)

Scenario

A simple webpage should greet users based on the name they enter. This helps beginners understand how JavaScript functions accept parameters and how variables behave inside functions.

Requirements

- Create an input box to enter a user's name.
- Create a button labeled "Generate Greeting".
- When the button is clicked:
- A JavaScript function should accept the name as a parameter.
- Display a greeting message like:
"Hello, Rahul! Welcome to our website."
- Display the greeting inside a <p> element.

Technical Constraints

- Use only vanilla JavaScript (no libraries).
- Use function keyword (not arrow functions for this task).
- Use document.getElementById() for DOM manipulation.
- Variable for the greeting message must be declared inside the function.

Learning Outcome

You should be able to:

- Understand how functions receive input using parameters.
- Learn local variable scope.
- Manipulate webpage elements using the DOM.

Code:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Greeting Generator</title>
```

```
</head>
```

```
<body>
```

```
<h2>Greeting Generator</h2>
```

```
<!-- Input box -->
```

```
<input type="text" id="username" placeholder="Enter your name">
```

```
<!-- Button -->
```

```
<button onclick="generateGreeting()">Generate Greeting</button>
```

```
<!-- Output -->
```

```
<p id="greetingMessage"></p>
```

```
<script>
```

```
function generateGreeting() {
```

```
    // Get input value
```

```
    var name = document.getElementById("username").value;
```

```
    // Local variable inside function
```

```
    var message = "Hello, " + name + "! Welcome to our website.";
```

```
    // Display greeting
```

```
    document.getElementById("greetingMessage").innerText = message;
```

```
}
```

```
</script>
```

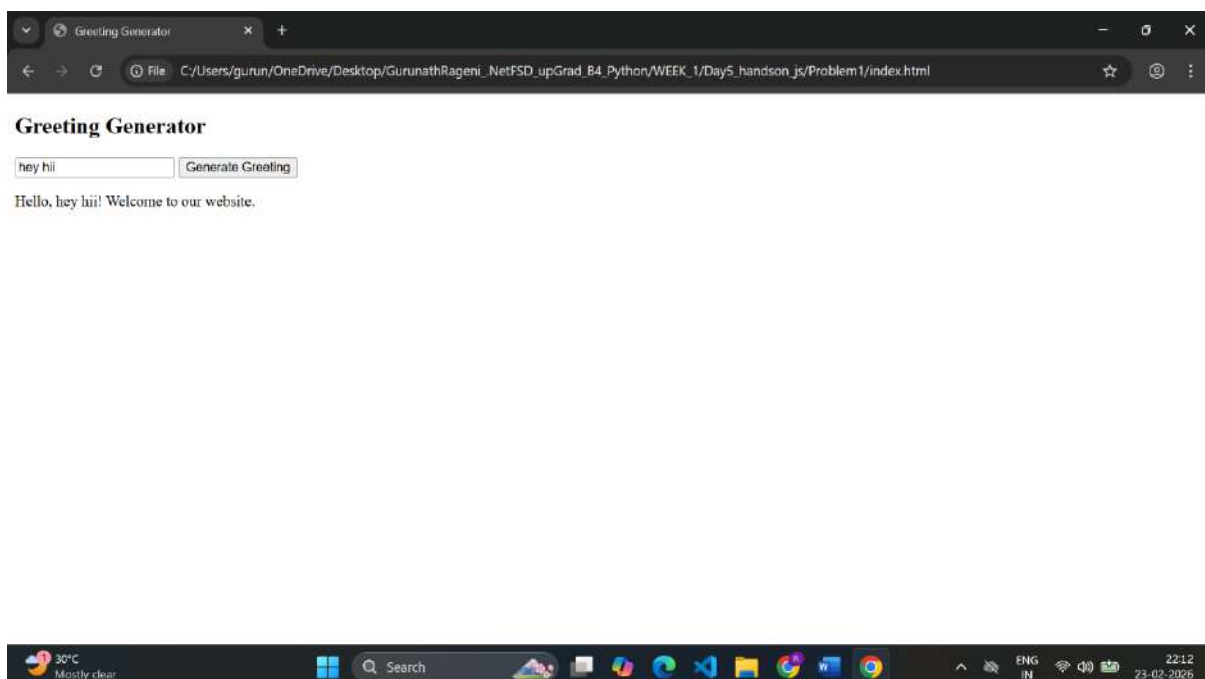
```
</body>
```

```
</html>
```

Code Screenshot:

```
WEEK_1 > Day5_handson_js > Problem1 > <> index.html
 2  <html>
 6  <body>
13  <!-- Button -->
14  <button onclick="generateGreeting()">Generate Greeting</button>
15
16  <!-- Output -->
17  <p id="greetingMessage"></p>
18
19  <script>
20
21  function generateGreeting() {
22
23      // Get input value
24      var name = document.getElementById("username").value;
25
26      // Local variable inside function
27      var message = "Hello, " + name + "! Welcome to our website.";
28
29      // Display greeting
30      document.getElementById("greetingMessage").innerText = message;
31  }
32
33  </script>
34
35  </body>
36  </html>
```

Output Screenshot:



Explanation:

This webpage demonstrates basic DOM manipulation using vanilla JavaScript to capture user input through `document.getElementById()`. When the button is clicked, the `generateGreeting()` function executes and retrieves the entered name as input. Inside the function, a local variable is declared to construct the greeting message, illustrating the concept of function scope. The generated message is then dynamically displayed in a `<p>` element, showing how JavaScript updates webpage content in real time.

Problem 2: Simple Object-Based User Info Display (Level-1)

Scenario

A webpage needs to display basic user information stored inside a JavaScript object.

Requirements

Create a JavaScript object `user` with properties:

- `name`
- `age`
- `city`

Create a function `displayUserInfo(userObj)`:

- Accepts the object as a parameter.
- Displays user details in separate `<p>` elements.

A button click should trigger the function.

Technical Constraints

- Object properties must be accessed using dot notation.
- Function should not use global variables.
- DOM elements should be updated dynamically.

Code:

```
<!DOCTYPE html>

<html>

<head>

  <title>User Info Display</title>

</head>

<body>
```

```
<h2>User Information</h2>
```

```
<button onclick="showUser()">Display User Info</button>
```

```
<p id="name"></p>
```

```
<p id="age"></p>
```

```
<p id="city"></p>
```

```
<script>
```

```
// Create user object
```

```
var user = {  
  name: "Rahul",  
  age: 22,  
  city: "Hyderabad"  
};
```

```
// Function that accepts object as parameter
```

```
function displayUserInfo(userObj) {  
  
  // Access using dot notation  
  document.getElementById("name").innerText = "Name: " + userObj.name;  
  document.getElementById("age").innerText = "Age: " + userObj.age;  
  document.getElementById("city").innerText = "City: " + userObj.city;  
}
```

```
// Button trigger function
```

```
function showUser() {  
  displayUserInfo(user);  
}
```

</script>

</body>

</html>

Code screenshot:

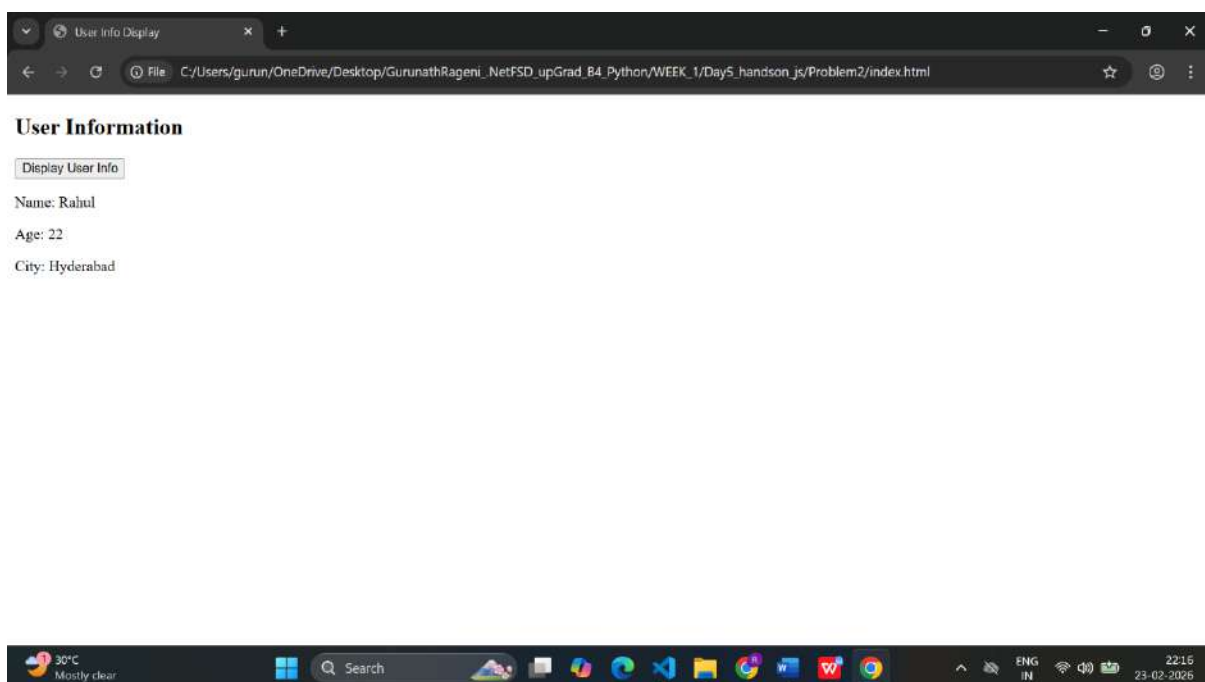
```
WEEK_1 > Day5_handson_js > Problem2 > <> index.html > html
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>User Info Display</title>
5  </head>
6  <body>
7
8  <h2>User Information</h2>
9
10 <button onclick="showUser()">Display User Info</button>
11
12 <p id="name"></p>
13 <p id="age"></p>
14 <p id="city"></p>
15
16 <script>
17
18 // Create user object
19 var user = {
20 |   name: "Rahul",
21 |   age: 22,
22 |   city: "Hyderabad"
23 | };
24
25 // Function that accepts object as parameter
```

```

25 // Function that accepts object as parameter
26 function displayUserInfo(userObj) {
27
28     // Access using dot notation
29     document.getElementById("name").innerText = "Name: " + userObj.name;
30     document.getElementById("age").innerText = "Age: " + userObj.age;
31     document.getElementById("city").innerText = "City: " + userObj.city;
32 }
33
34 // Button trigger function
35 function showUser() {
36     displayUserInfo(user);
37 }
38
39 </script>
40
41 </body>
42 </html>

```

Output Screenshot:



Explanation:

This webpage uses a JavaScript object to store structured user information such as name, age, and city. The `displayUserInfo(userObj)` function accepts the object as a parameter and accesses its properties using dot notation. It dynamically updates the `<p>` elements through DOM manipulation when the button is clicked.

Problem 3: Counter App with Scope Control (Level-2)

Scenario

Build a counter application where users can increment and reset a value. This problem focuses on variable scope and DOM manipulation.

🔴 Requirements

Display a counter value starting from 0.

Create two buttons:

- **Increment**
- **Reset**

Use:

- A global variable to store counter value.
- A function `incrementCounter(step)` that:
 - Accepts step value as a parameter.
 - Updates the counter.

Reset button should reset the counter to 0.

🔧 Technical Constraints

- Counter value must be maintained outside the function (global scope).
- DOM updates must happen inside functions only.
- No inline JavaScript in HTML.

Code:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Counter App</title>
```

```
</head>
```

```
<body>
```

```
<h2>Counter App</h2>
```

```
<p id="counterValue">0</p>
```



```
<button id="incrementBtn">Increment</button>
```

```
<button id="resetBtn">Reset</button>
```

```
<script>
```

```
// ✅ Global variable (outside functions)
```

```
var counter = 0;
```

```
// Function to increment counter
```

```
function incrementCounter(step) {
```

```
    counter += step;
```

```
    document.getElementById("counterValue").innerText = counter;
```

```
}
```

```
// Function to reset counter
```

```
function resetCounter() {
```

```
    counter = 0;
```

```
    document.getElementById("counterValue").innerText = counter;
```

```
}
```

```
// Event listeners (No inline JS)
```

```
document.getElementById("incrementBtn").addEventListener("click", function() {
```

```
    incrementCounter(1); // Passing step value as parameter
```

```
});
```

```
document.getElementById("resetBtn").addEventListener("click", resetCounter);
```

```
</script>
```

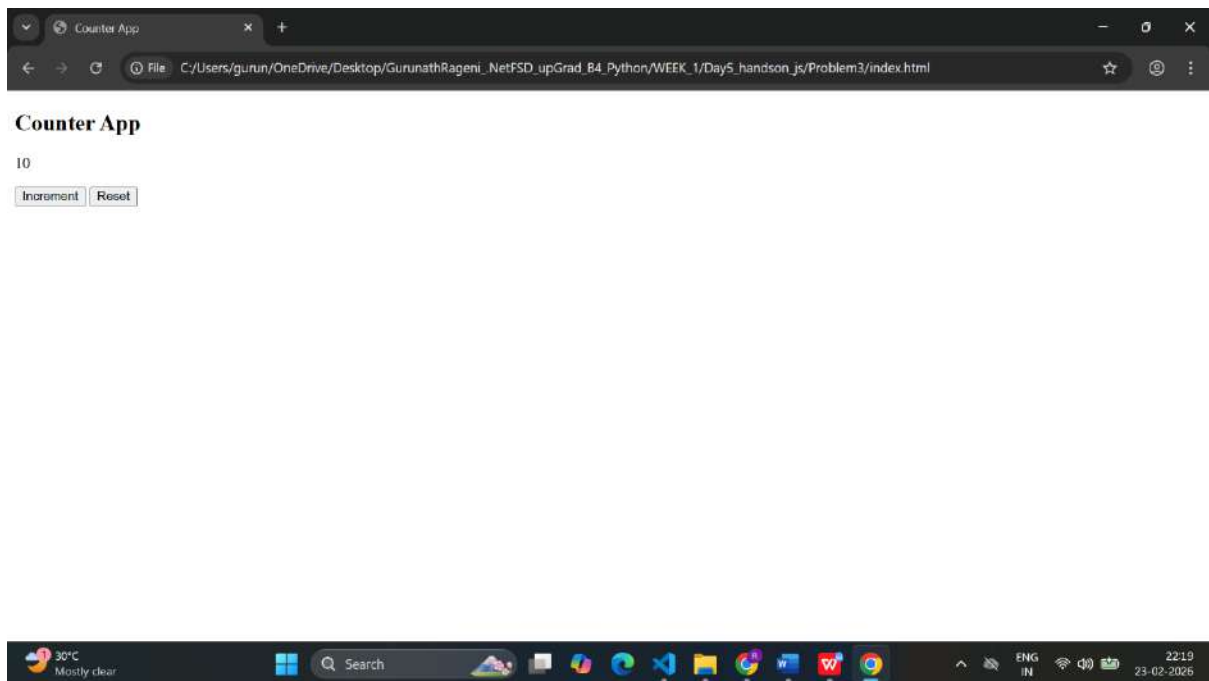
```
</body>
```

```
</html>
```

Code screenshot:

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Counter App</title>
5  </head>
6  <body>
7
8  <h2>Counter App</h2>
9
10 <p id="counterValue">0</p>
11
12 <button id="incrementBtn">Increment</button>
13 <button id="resetBtn">Reset</button>
14
15 <script>
16
17 // Global variable (outside functions)
18 var counter = 0;
19
20 // Function to increment counter
21 function incrementCounter(step) {
22 |   counter += step;
23 |   document.getElementById("counterValue").innerText = counter;
24 | }
25
26 // Function to reset counter
27 function resetCounter() {
28 |   counter = (method) Document.getElementById(elementId: string): HTMLElement | null
29 |   document.getElementById("counterValue").innerText = counter;
30 | }
31
32 // Event listeners (No inline JS)
33 document.getElementById("incrementBtn").addEventListener("click", function() {
34 |   incrementCounter(1); // Passing step value as parameter
35 | });
36
37 document.getElementById("resetBtn").addEventListener("click", resetCounter);
38
39 </script>
40
41 </body>
42 </html>
```

Output Screenshot:



Explanation:

This counter application maintains the counter value in a global variable to preserve state across function calls. The `incrementCounter(step)` function accepts a parameter to update the value dynamically, demonstrating controlled scope usage. DOM updates are handled strictly inside functions, while event listeners are attached using `addEventListener()` instead of inline JavaScript.

Problem 4: Dynamic Student Profile Manager (Level-2)

Scenario

Create a mini student profile system where details are stored in an object and displayed dynamically using DOM manipulation.

📌 Requirements

Create a student object with:

- name
- rollNo
- marks

Create a function `updateStudentProfile(studentObj)`:

- Accepts the object as a parameter.
- Displays details in a styled `<div>`.

Add another function `updateMarks(newMarks)`:

- Updates marks and refreshes the display.

Technical Constraints

- Student object must be declared in global scope.
- Functions should update object values using parameters.
- DOM should update without page refresh.

Code:

```
<!DOCTYPE html>

<html>

<head>

  <title>Dynamic Student Profile Manager</title>

  <style>

    #profileCard {

      width: 300px;

      padding: 15px;

      border: 2px solid #333;

      border-radius: 8px;

      background-color: #f4f4f4;

      margin-top: 15px;

    }

  </style>

</head>

<body>


  <h2>Student Profile Manager</h2>


  <button id="showProfileBtn">Show Profile</button>

  <button id="updateMarksBtn">Update Marks (+10)</button>


  <div id="profileCard"></div>
```

<script>

//  Global student object

```
var student = {  
  name: "Rahul",  
  rollNo: 101,  
  marks: 75  
};
```

// Function to display/update profile

```
function updateStudentProfile(studentObj) {
```

```
  var profileDiv = document.getElementById("profileCard");
```

```
  profileDiv.innerHTML =
```

```
    "<h3>Student Profile</h3>" +
```

```
    "<p><strong>Name:</strong> " + studentObj.name + "</p>" +
```

```
    "<p><strong>Roll No:</strong> " + studentObj.rollNo + "</p>" +
```

```
    "<p><strong>Marks:</strong> " + studentObj.marks + "</p>";
```

```
}
```

// Function to update marks and refresh display

```
function updateMarks(newMarks) {
```

```
  student.marks = newMarks; // Update global object
```

```
  updateStudentProfile(student); // Refresh UI
```

```
}
```

// Event listeners (No inline JS)

```
document.getElementById("showProfileBtn")
```

```

        .addEventListener("click", function() {
            updateStudentProfile(student);
        });

document.getElementById("updateMarksBtn")

        .addEventListener("click", function() {
            var updatedMarks = student.marks + 10;
            updateMarks(updatedMarks);
        });
</script>
</body>
</html>

```

Code Screenshots:

```

WEEK_1 > Day5_handson_js > Problem4 > index.html > html > body > script
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Dynamic Student Profile Manager</title>
5      <style>
6          #profileCard {
7              width: 300px;
8              padding: 15px;
9              border: 2px solid #333;
10             border-radius: 8px;
11             background-color: #f4f4f4;
12             margin-top: 15px;
13         }
14     </style>
15 </head>
16 <body>
17
18     <h2>Student Profile Manager</h2>
19
20     <button id="showProfileBtn">Show Profile</button>
21     <button id="updateMarksBtn">Update Marks (+10)</button>
22
23     <div id="profileCard"></div>
24
25     <script>
26
27     // Global student object
28     var student = {
29         name: "Rahul",

```

```

29     name: "Rahul",
30     rollNo: 101,
31     marks: 75
32 };
33
34 // Function to display/update profile
35 function updateStudentProfile(studentObj) {
36
37     var profileDiv = document.getElementById("profileCard");
38
39     profileDiv.innerHTML =
40         "<h3>Student Profile</h3>" +
41         "<p><strong>Name:</strong> " + studentObj.name + "</p>" +
42         "<p><strong>Roll No:</strong> " + studentObj.rollNo + "</p>" +
43         "<p><strong>Marks:</strong> " + studentObj.marks + "</p>";
44 }
45
46 // Function to update marks and refresh display
47 function updateMarks(newMarks) {
48
49     student.marks = newMarks; // Update global object
50     updateStudentProfile(student); // Refresh UI
51 }
52
53 // Event listeners (No inline JS)

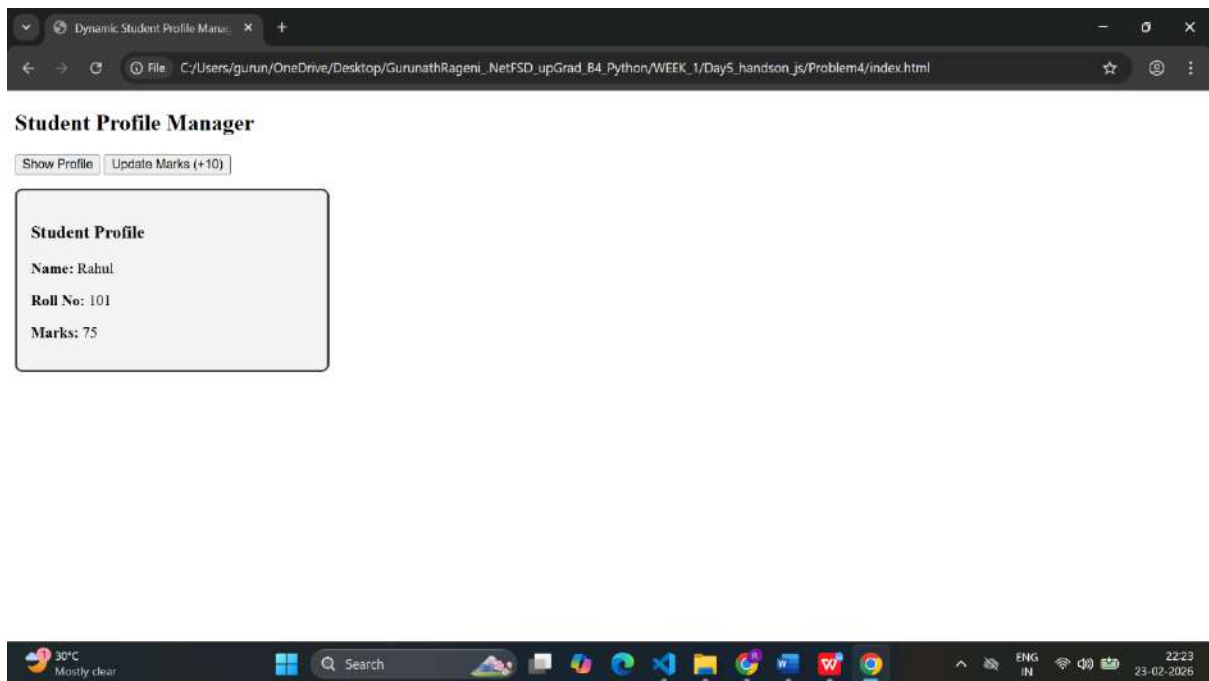
```

```

53 // Event listeners (No inline JS)
54 document.getElementById("showProfileBtn")
55     .addEventListener("click", function() {
56         updateStudentProfile(student);
57     });
58
59 document.getElementById("updateMarksBtn")
60     .addEventListener("click", function() {
61         var updatedMarks = student.marks + 10;
62         updateMarks(updatedMarks);
63     });
64
65 </script>
66
67 </body>
68 </html>

```

Output Screenshot:



Explanation:

This application maintains a global student object to preserve structured profile data across functions. The `updateStudentProfile(studentObj)` function dynamically renders object properties into a styled `<div>` using DOM manipulation. The `updateMarks(newMarks)` function updates the object's marks property via parameter input and refreshes the UI without reloading the page.